Difusão condicional no projeto

Resumo da conversa, próximos passos e comandos de uso · 12 de novembro de 2026

Principais pontos

- O modelo de difusão original é **incondicional**: gera peças aleatórias do Fashion-MNIST, sem escolher a classe.
- Foi criada uma **difusão condicional**, um modelo separado com treino próprio, que recebe a classe e permite pedir, por exemplo, “tênis”.
- Ela usa **classifier-free guidance**, controlado pelo parâmetro `guidance_scale`.
- Mesmo com 50 épocas, as imagens saíam saturadas. A causa era o amostrador: a **limitação de x_0 a [-1, 1]** foi incorporada ao projeto e resolve sem treinar de novo.
- Próximo passo recomendado: U-Net e EMA, para deixar as imagens mais nítidas.

1. Como o modelo de difusão funciona

- **Treino:** a imagem real recebe uma quantidade sorteada de ruído, e a rede aprende a **prever qual ruído foi adicionado**.
- **Geração:** começa de **ruído puro**, e a rede remove o ruído aos poucos, em vários passos (200 no baseline), até sobrar uma imagem.
- O ruído não é aplicado sobre algo já aprendido: ele é o ponto de partida, e o modelo “esculpe” a imagem a partir dele.

2. Difusão incondicional x condicional

Aspecto	Difusão (incondicional)	Difusão condicional
Entrada da rede	Imagem ruidosa + passo de tempo	Imagem ruidosa + passo de tempo + **classe**
Rótulos no treino	Descartados (`for images, _`)	Usados
O que gera	Uma peça aleatória do dataset	A classe pedida (ex.: tênis)
Treino	`train_diffusion.py`	`train_conditional_diffusion.py`
Gerar após o treino	Não há comando separado	`generate_conditional_diffusion.py`

- **É um modelo separado, com treino próprio.** Ele tem uma peça a mais (o embedding da classe) e precisa ver os rótulos durante o treino para associar “tênis” à forma de um tênis. Os checkpoints dos dois modelos não são intercambiáveis.
- Como 10% do treino é feito sem classe, o modelo condicional **também aprende a gerar sem classe**. Com `--guidance-scale 0` ele se comporta como o incondicional, então pode cobrir os dois casos no futuro.
- **Ligação com a pesquisa:** a pergunta de pesquisa prevê um modelo “condicionado por achados radiológicos”. O mecanismo é o mesmo: em vez de “tênis”, as classes seriam achados como “pneumonia”, “derrame pleural” ou “normal” (rótulos do CheXpert).

3. Como o condicionamento funciona

- Um **embedding da classe** é somado ao embedding do passo de tempo. Assim, cada bloco residual da rede recebe as duas informações.
- **Classifier-free guidance:** no treino, uma fração das imagens (`label_dropout`, 10%) é apresentada sem classe, e a mesma rede aprende a previsão com classe e sem classe. Na geração, as duas são combinadas:

$$\text{ruído} = \text{ruído}_{\text{sem classe}} + \text{guidance_scale} \times (\text{ruído}_{\text{com classe}} - \text{ruído}_{\text{sem classe}})$$

`guidance_scale`	Efeito
0	Ignora a classe (equivale ao modelo incondicional).
1	Usa só a previsão com classe, sem reforço.
Maior que 1	Reforça a classe pedida, com menos diversidade (3 no baseline).
Muito alto	Pode saturar as imagens; a limitação de x_0 (seção 8) reduz esse efeito.

4. Arquivos criados e alterados

Arquivo	Papel
`scripts/conditional_diffusion_model.py`	Modelo: rede com embedding de classe e classifier-free guidance.
`scripts/train_conditional_diffusion.py`	Treino e registro do experimento (inclui a contagem de imagens por classe).
`scripts/generate_conditional_diffusion.py`	Gera as classes escolhidas a partir de um checkpoint treinado.
`tests/test_conditional_diffusion.py`	11 testes automatizados novos.
`scripts/diffusion_model.py`	Passo de geração com a opção `clip_denoised` (desligada por padrão).
`scripts/train_diffusion.py`	Lê `clip_denoised` da configuração e registra no metadata e no checkpoint.
`configs/baseline.yml`	Nova seção `conditional_diffusion`; `clip_denoised: true` nas duas difusões.
`scripts/train_utils.py`	`save_samples` aceita `nrow` (sem efeito nos outros modelos).
`README.md`	Documentação da difusão condicional e da limitação de x_0.

Na difusão incondicional, só a geração mudou, e apenas quando `clip_denoised` está ligado: desligado, o resultado é idêntico ao anterior (há um teste para isso). A difusão latente não usa a opção. As alterações ainda **não foram commitadas**.

5. Classes do Fashion-MNIST

No script de geração, as classes podem ser informadas pelo nome, com ou sem acento e maiúsculas, ou pelo índice. “Sapato” corresponde a três classes: **5 (sandália), 7 (tênis) e 9 (bota)**.

Índice	Nome no script	Nome oficial	Índice	Nome no script	Nome oficial
0	`camiseta`	T-shirt/top	5	`sandalia`	Sandal
1	`calca`	Trouser	6	`camisa`	Shirt
2	`pulover`	Pullover	7	`tenis`	Sneaker
3	`vestido`	Dress	8	`bolsa`	Bag
4	`casaco`	Coat	9	`bota`	Ankle boot

6. Comandos

Todos os comandos são executados no PowerShell, na raiz do projeto.

6.1 Preparar o ambiente e testar

```
# Ativar o ambiente virtual do projeto
.\venv\Scripts\Activate.ps1

# Rodar os testes automatizados (CPU, dados artificiais, sem download)
python -m unittest discover -s tests -v
```

6.2 Treinar

```
# Teste rápido do fluxo (valida o funcionamento, não a qualidade)
python scripts\train_conditional_diffusion.py `
    --epochs 1 --samples 256 --batch-size 32 --generated-samples 1 --deterministic

# Treino completo do baseline (50 épocas; cerca de 35 minutos na GPU atual)
python scripts\train_conditional_diffusion.py --deterministic

# Difusão incondicional, para comparação
python scripts\train_diffusion.py --deterministic
```

Ao fim do treino, o terminal mostra a pasta da execução, no formato `outputs\conditional_diffusion\<identificador>\`. Lá ficam o `checkpoint.pt`, as amostras de cada época (`samples_epoch_XX.png`), o `metrics.csv` e o `metadata.json`.

6.3 Gerar imagens de classes escolhidas

```
# Defina o checkpoint com UMA das duas linhas abaixo
# a) o do treino de 50 épocas já concluído
$ckpt = "outputs\conditional_diffusion\20261112T233335_375850Z_8d91b0ef\checkpoint.pt"

# b) o de um treino novo: troque <identificador> INTEIRO (inclusive < >)
#    pelo nome da pasta impresso no terminal
$ckpt = "outputs\conditional_diffusion\<identificador>\checkpoint.pt"

# Sapatos: 8 de cada classe (sandália, tênis, bota), uma linha por classe
python scripts\generate_conditional_diffusion.py $ckpt --classes sandalia tenis bota

# Mais imagens e outra variação aleatória
python scripts\generate_conditional_diffusion.py $ckpt --classes tenis --per-class 32 --seed 1

# Ajustar a força da classe, sem retreinar
python scripts\generate_conditional_diffusion.py $ckpt --classes bota --guidance-scale 1.5

# Escolher onde salvar o PNG
python scripts\generate_conditional_diffusion.py $ckpt --classes vestido --output vestidos.png
```

Opção	O que faz	Padrão
`--classes`	Nomes ou índices das classes; cada uma vira uma linha da grade.	Obrigatória
`--per-class`	Quantas imagens de cada classe.	8
`--seed`	Variação aleatória: outra semente gera outras imagens.	0
`--guidance-scale`	Força da classe (seção 3).	Valor do treino (3)
`--output`	Caminho do PNG gerado.	Pasta do checkpoint
`--no-clip-denoised`	Desliga a limitação de x_0 (o arquivo recebe o sufixo `_semclip`).	Limitação ligada

Se o caminho do checkpoint estiver errado (por exemplo, com `<pasta>` ou `<identificador>` no lugar do nome real), o script avisa e lista os checkpoints mais recentes. A pasta `outputs/` é ignorada pelo Git: para arquivar um resultado da pesquisa, preserve a pasta completa da execução.

7. Campos que controlam a quantidade de imagens

Atenção: o nome **samples** aparece em dois lugares, com significados diferentes.

Onde	Controla	Observação
`data.samples` / `--samples`	Imagens **de treino** (60.000).	**Não** mexa nisso para gerar mais imagens.
`models.conditional_diffusion.samples` / `--generated-samples`	Imagens **geradas por classe** ao fim de cada época (8, ou seja, 80 no total).	Só acompanha a evolução do treino.
`models.diffusion.samples`	Total de imagens geradas por época no modelo incondicional (16).	
`--per-class` e `--classes` (script de geração)	Imagens geradas **depois** do treino.	Forma recomendada; não precisa retreinar.

Seção do `configs/baseline.yml` usada pela difusão condicional (o número de épocas fica em `training.epochs`, hoje 50):

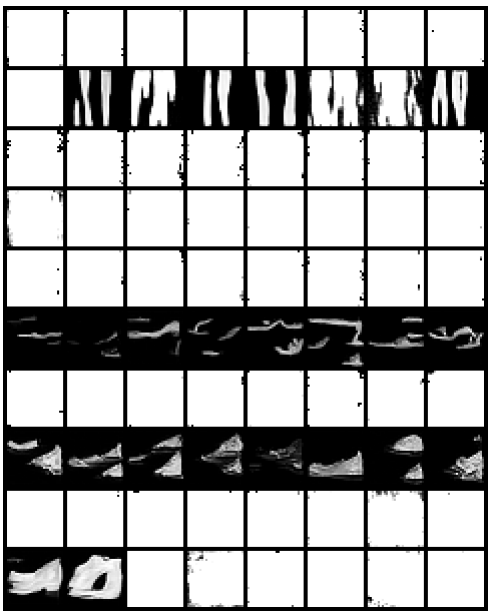
```
conditional_diffusion:
  learning_rate: 0.0002
  timesteps: 200
  channels: 64
  samples: 8           # imagens por classe nas amostras de cada época
  label_dropout: 0.1   # fração do treino sem classe (classifier-free guidance)
  guidance_scale: 3.0  # 1 = sem guidance; maior = classe mais evidente
  clip_denoised: true  # limita x0 a [-1, 1] na geração; evita imagens saturadas
```

Limitação atual: todas as imagens saem num único PNG em grade e são geradas de uma vez na GPU. Isso funciona bem para dezenas ou algumas centenas de imagens. Para gerar milhares em arquivos separados (por exemplo, um dataset sintético), o script precisaria gerar em lotes.

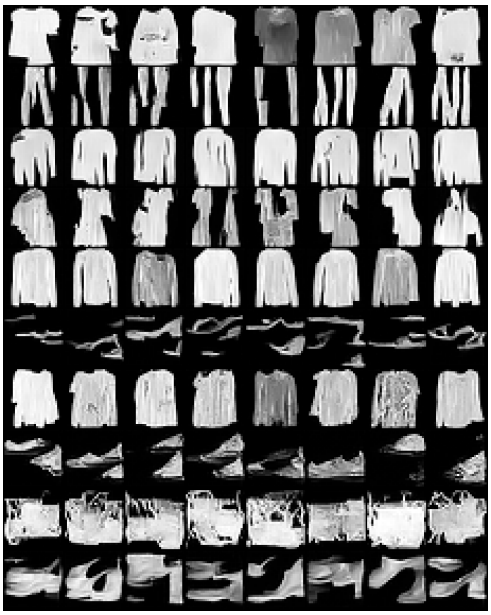
8. Verificação e correção da saturação

- **Testes:** os 16 testes automatizados passam (5 antigos e 11 novos).
- **Treino de 50 épocas** (60.000 imagens, GPU, modo determinístico): concluído em 34 minutos. O erro de previsão de ruído caiu de 0,1405 (época 1) para 0,0740 (época 50) e quase estabilizou: era 0,0751 na época 40.
- **Problema:** mesmo com 50 épocas, as imagens saíam **saturadas** (quadrados todos brancos ou pretos), principalmente nas classes com muita área clara, como camiseta, casaco e bolsa. O guidance amplifica o efeito.
- **Causa e correção:** em cada passo, a rede permite estimar a imagem limpa (x_0). Sem limite, os erros se acumulam e a imagem satura. Limitar essa estimativa a [-1, 1], uma técnica do DDPM original, resolveu **com o mesmo checkpoint, sem treinar de novo**. A correção agora faz parte do projeto (`clip_denoised`).

Sem limitação de x_0 *(amostrador antigo)*



Com limitação de x_0 *(padrão atual)*



Uma linha por classe, na ordem dos índices: camiseta, calça, pulôver, vestido, casaco, sandália, camisa, tênis, bolsa e bota. Mesmo checkpoint (50 épocas), mesma semente, guidance 3. Imagens de 28×28 pixels ampliadas 3 vezes.

9. Como melhorar a qualidade e a nitidez

Em ordem de impacto e custo:

Sem mudar o código

1. **Ajustar `--guidance-scale` na geração.** Não precisa retreinar. Gere a mesma classe com alguns valores (por exemplo, 1, 2, 3 e 5) e compare: valores maiores deixam a classe mais evidente, com menos variedade.
2. **Gerar com outras sementes** (`--seed`) e escolher as melhores amostras para cada atividade.

Treinar mais épocas deve ajudar pouco agora: o erro quase estabilizou entre as épocas 40 e 50. O limite principal é a rede.

Com mudanças no código

3. **Trocar a rede por uma U-Net.** É a principal limitação atual: a rede tem só 2 blocos e trabalha sempre em 28×28, então cada pixel de saída "enxerga" apenas uma janela de 13×13 pixels e nunca vê a peça inteira. É por isso que alguns tênis saem "partidos" ou duplicados. A U-Net reduz e depois amplia a resolução para enxergar a imagem toda; é a arquitetura padrão do DDPM.
4. **EMA dos pesos:** gerar com uma média móvel dos pesos do treino. É padrão em difusão e deixa as amostras mais limpas, quase sem custo.
5. **Mais `timesteps`** (200 para 1000, como no DDPM original). O efeito é menor, e a geração fica 5 vezes mais lenta.

> **Limite do dataset:** o Fashion-MNIST tem imagens de 28×28 pixels. Mesmo um modelo perfeito gera imagens pequenas, e ampliar não acrescenta detalhe. A resolução vai importar nas radiografias, e é aí que entra a difusão latente que o projeto já tem.

10. Próximos passos

1. **Implementar U-Net e EMA**, como opções registradas no `metadata.json`, para manter a comparação com as execuções anteriores.
2. **Treinar de novo e comparar** a difusão condicional com a incondicional, as duas com a limitação de x_0 ligada.
3. **Commitar as alterações**, que ainda não estão versionadas.
4. **Depois:** adaptar o pipeline às radiografias de tórax, usando os achados do CheXpert como classes.